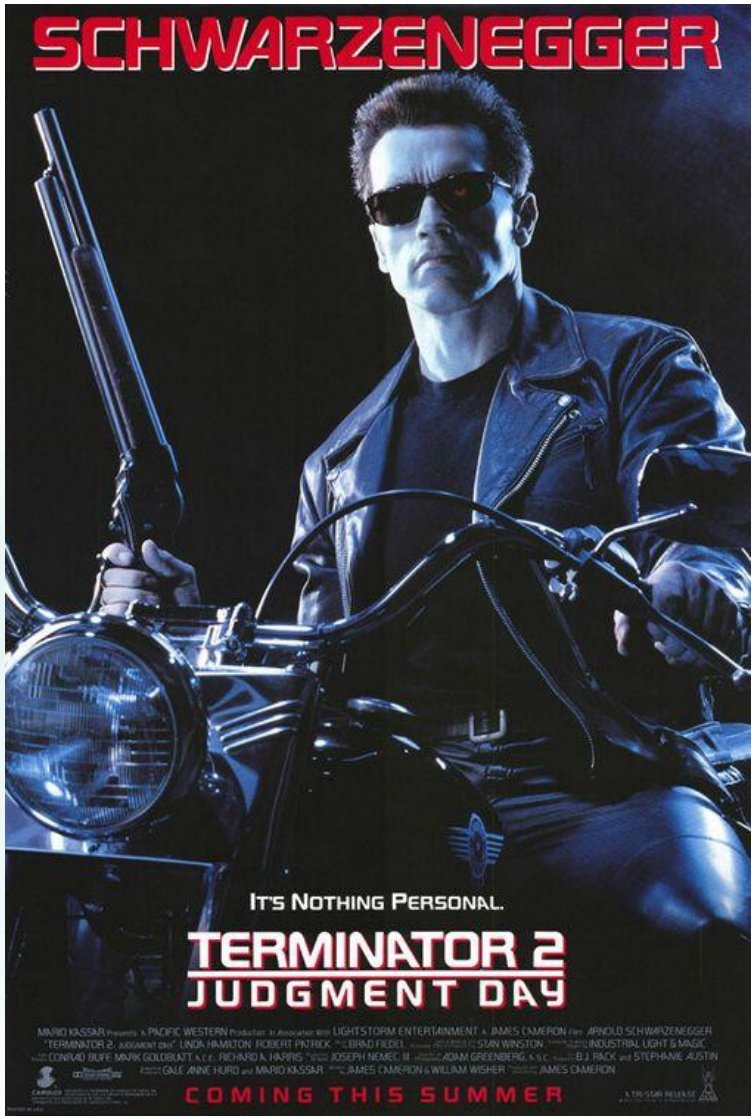
Neural Networks

Yu Sheng

Center for Statistical Science

Neural Networks in Sci-Fi



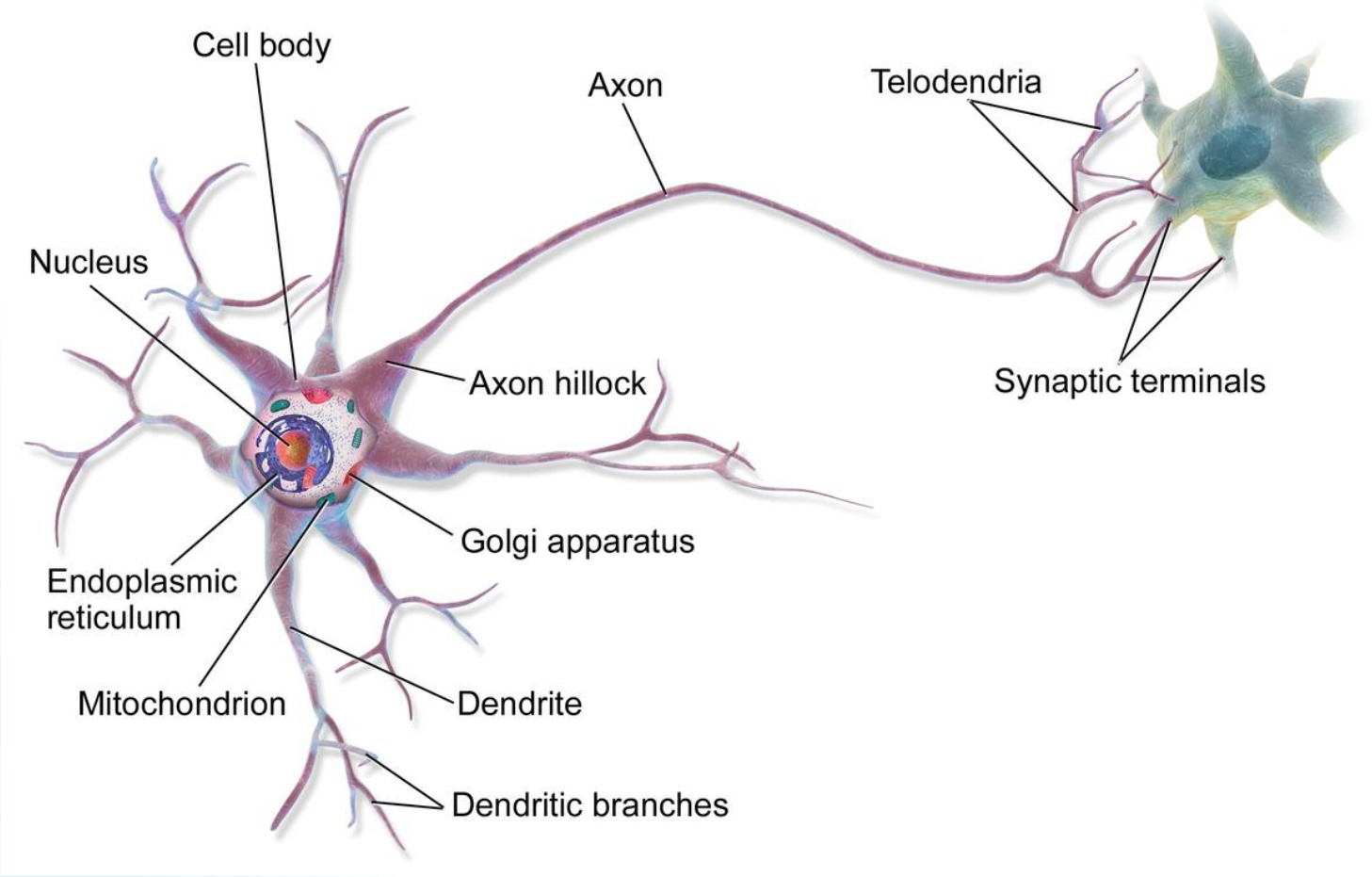
John Connor: Can you learn stuff you haven't been programmed with so you could be… you know, more human? And not such a dork all the time?

The Terminator: My CPU is a neural-net processor; a learning computer. But Skynet presets the switch to read-only when we're sent out alone.

Sarah Connor: Doesn't want you doing too much thinking, huh?

The Terminator: No.

Biological Neurons



- By BruceBlaus - Own work, CC BY 3.0, https://commons.wikimedia.org/w/index.php?curid=28761830

清华大学统计学研究中心

Biological Neurons

- Human brain: 100,000,000,000 neurons
- Each neuron receives input from 1,000 others
- Impulses arrive simultaneously
- Added together: an impulse can either increase or decrease the possibility of nerve pulse firing
- If sufficiently strong, a nerve pulse is generated
- The pulse forms the input to other neurons
- The interface of two neurons is called a synapse

清华大学统计学研究中心

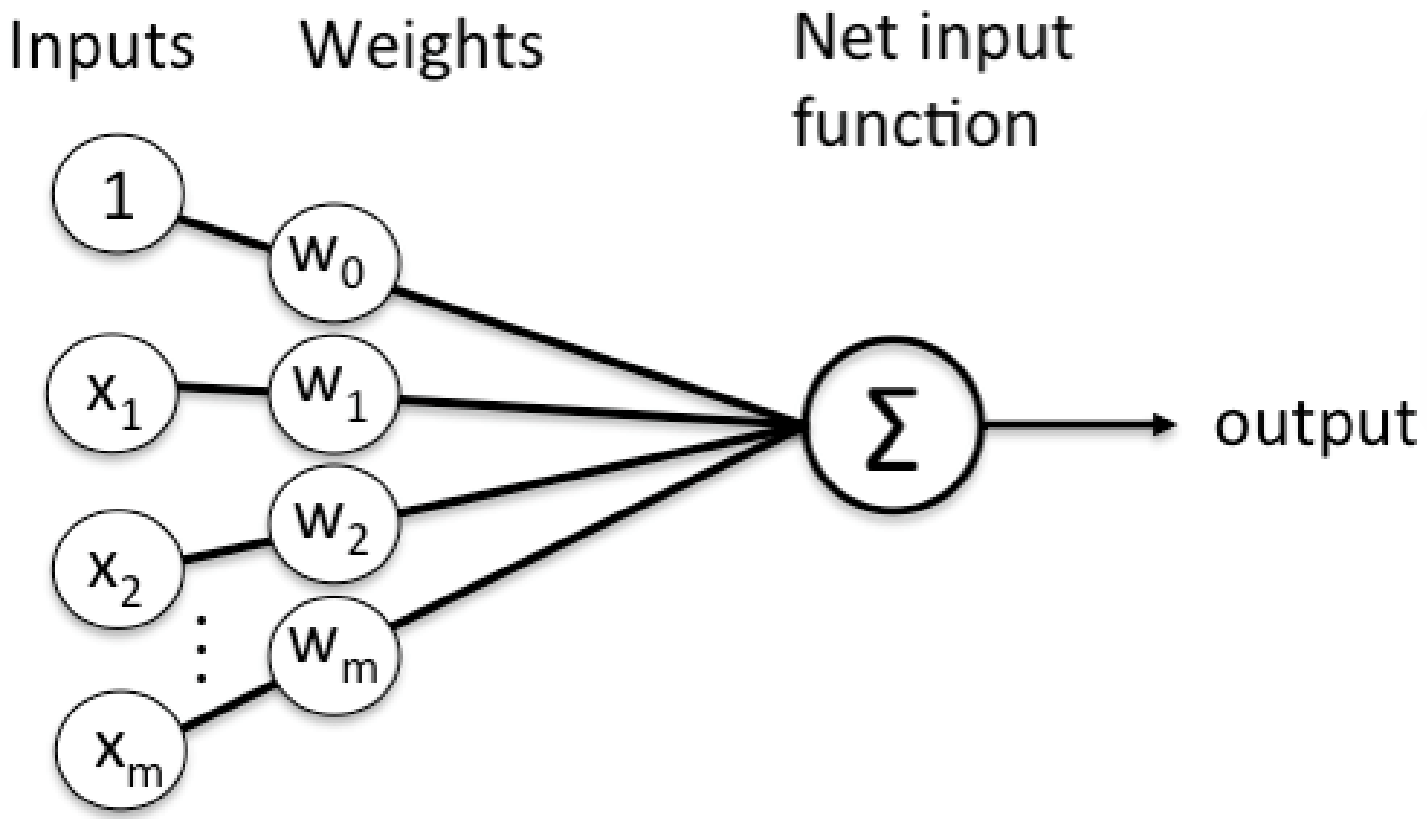
Linear Perceptrons

- Perceptron: a mathematical model for a single neuron
- Input: $x_1, \ldots, x_p$ (signal from other neurons)
- Weights: $w_1, \ldots, w_p$ (dendrites, can be negative)
- Optional constant term $x_0 = 1$ with weight w_0
- Activation function: linear (for the time being)

$$y = g\left(w_0 x_0 + w_1 x_1 + \cdots + w_p x_p\right)$$

$$= w_0 x_0 + w_1 x_1 + \cdots + w_p x_p$$

Linear Perceptrons



Learning in Linear Perceptrons

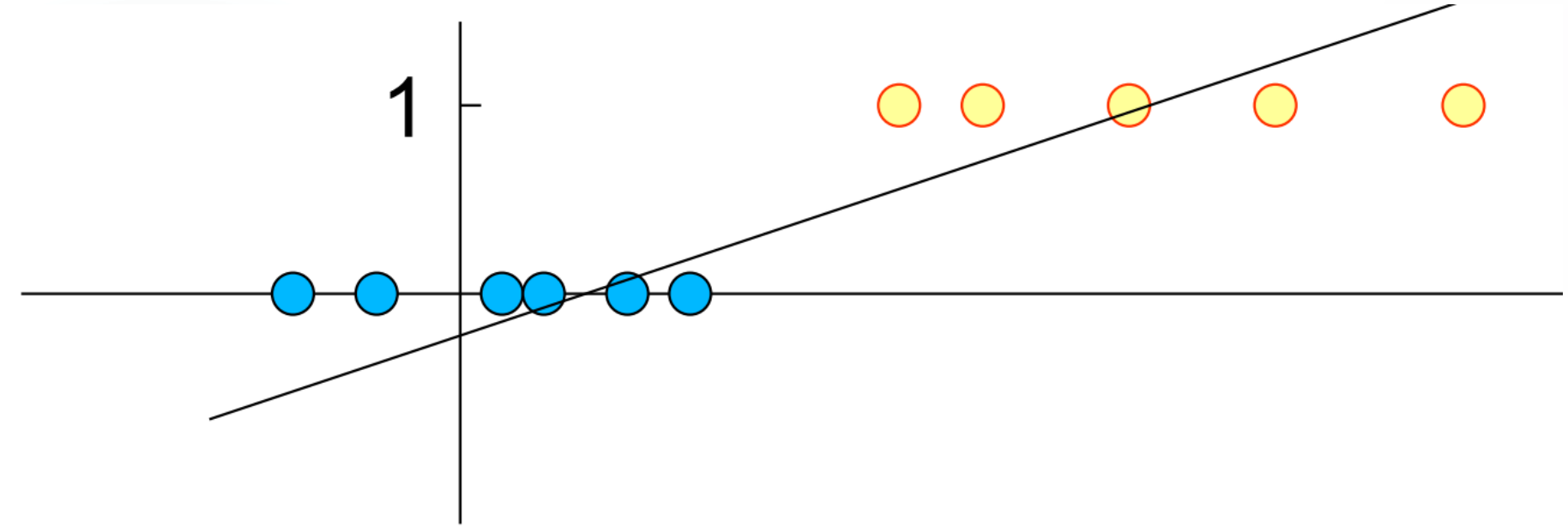
- Gradient descent: $w \leftarrow w - \alpha \nabla L(w)$
- α is a small constant, called the "learning rate"
- For regression problems, L could be the squared error loss, then for an observation (x, y),

$$w \leftarrow w - \alpha(w'x - y)x.$$

清华大学统计学研究中心

Limitation of Linear Perceptrons

- A linear perceptron is just a linear function
- For example, it won't be a nice fit to binary classifications



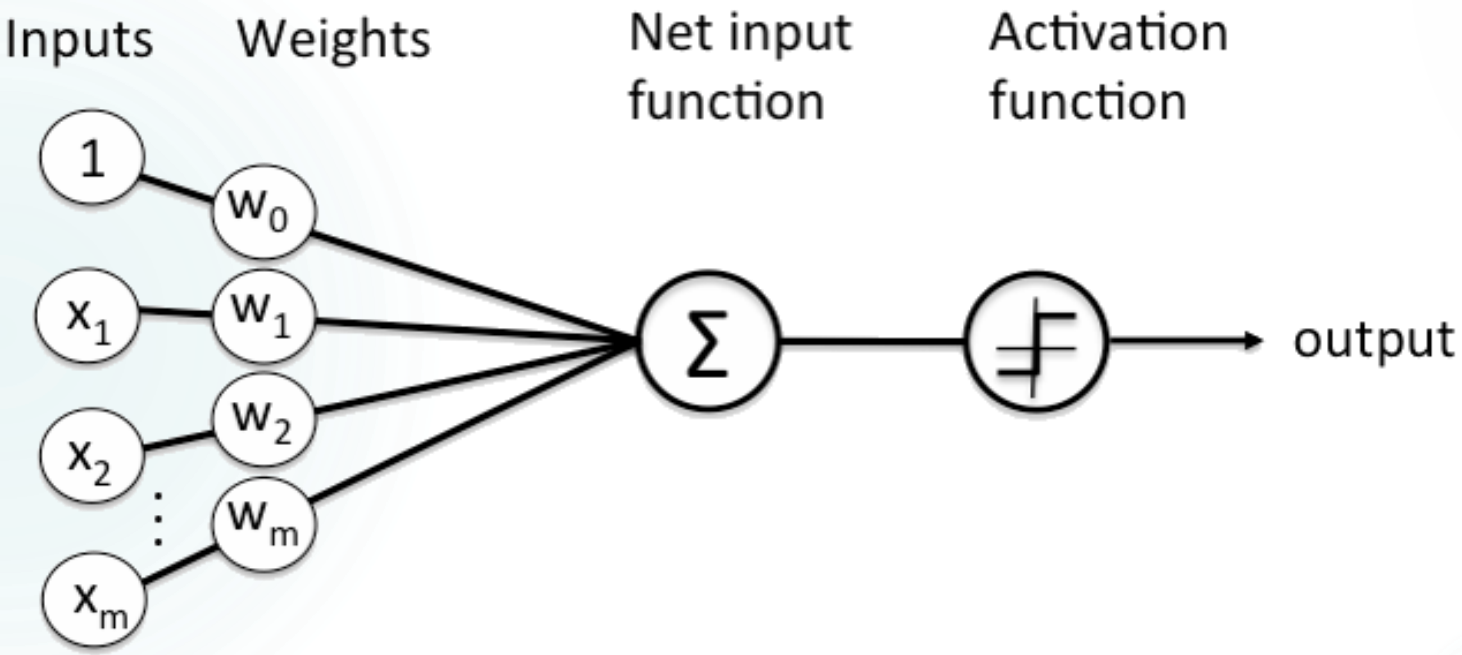
清华大学统计学研究中心

Nonlinear Perceptrons

- Change the activation function:

$$y = g(w_0 x_0 + w_1 x_1 + \cdots + w_p x_p)$$

where g is nonlinear.

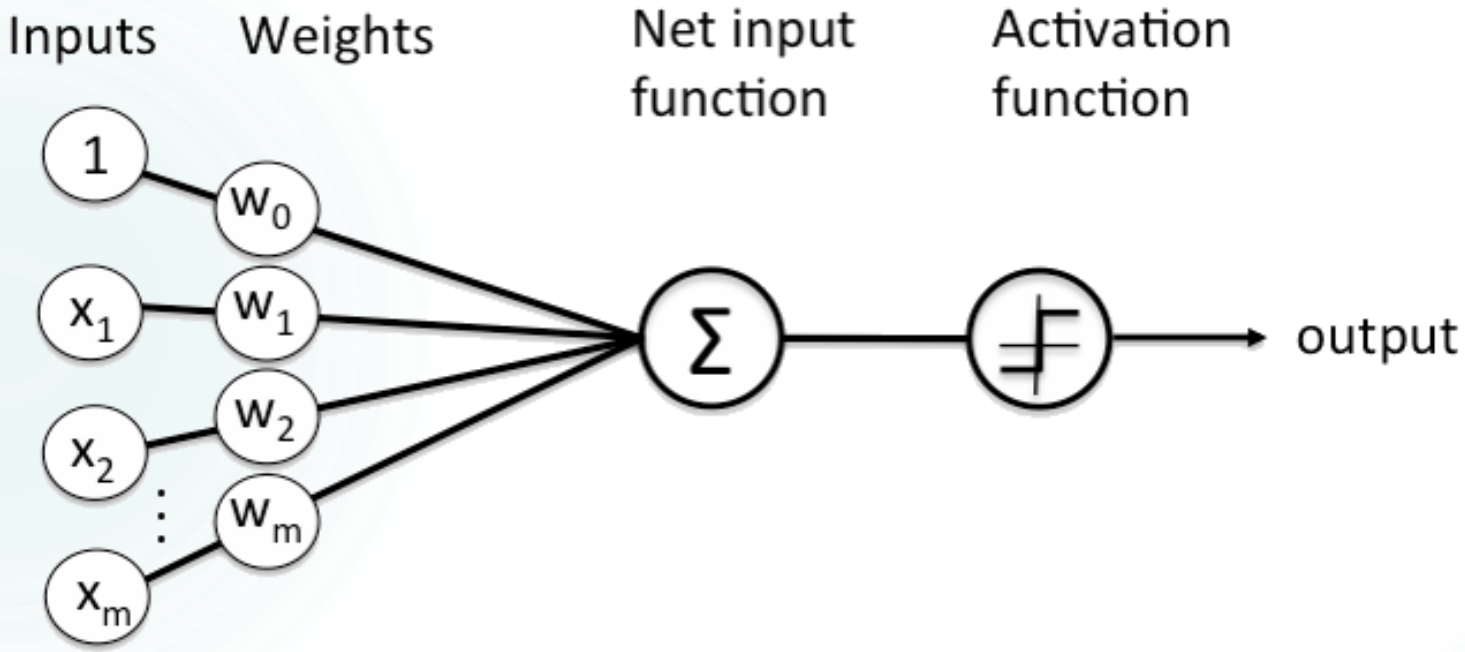


清华大学统计学研究中心

Nonlinear Perceptrons

- For example, g could be the step function (called the Linear Threshold Unit, or LTU)

$$g(x) = \begin{cases} 1, & x \geq 0 \\ 0, & x < 0 \end{cases}$$



清华大学统计学研究中心

LTU Perceptrons

- For example, g could be the step function (called the Linear Threshold Unit, or LTU)

$$g(x) = \begin{cases} 1, & x \geq 0 \\ 0, & x < 0 \end{cases}$$

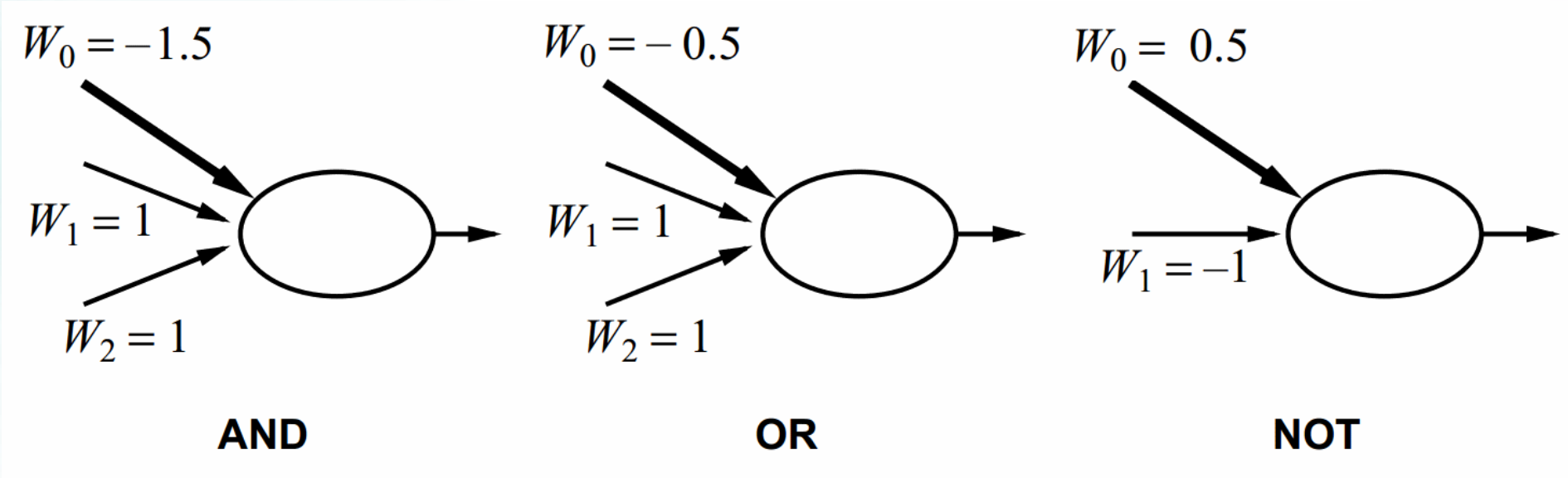
- Can you see how to make logic AND, OR, NOT with such a perceptron?

清华大学统计学研究中心

LTU Perceptrons

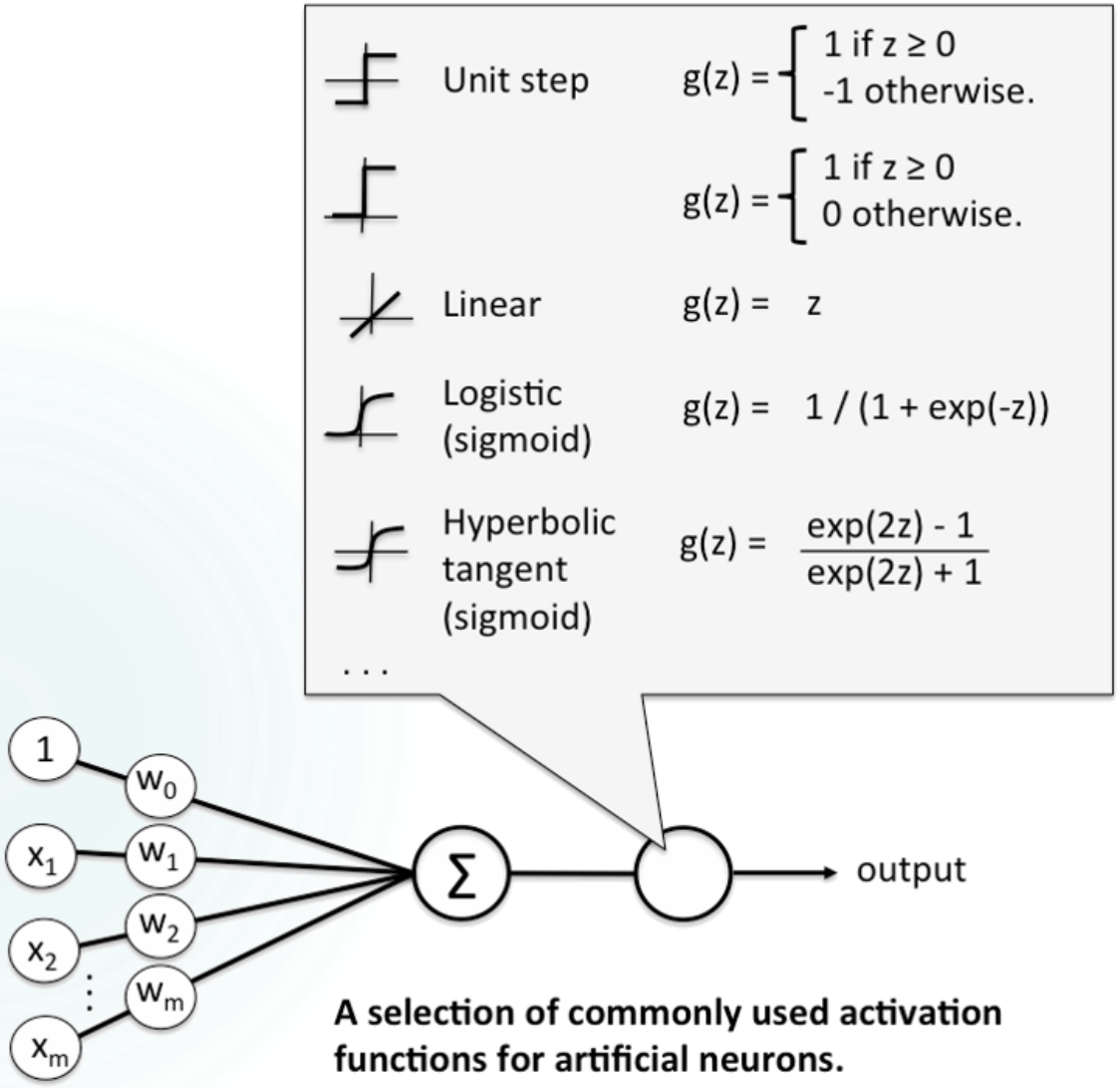
- For example, g could be the step function (called the Linear Threshold Unit, or LTU)

$$g(x) = \begin{cases} 1, & x \geq 0 \\ 0, & x < 0 \end{cases}$$



清华大学统计学研究中心

Other Activation Functions

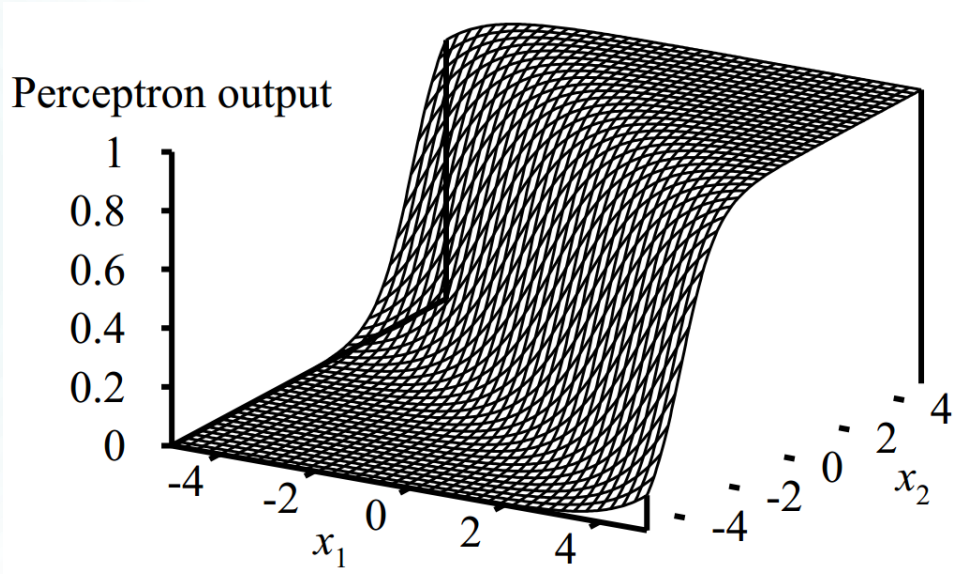


A selection of commonly used activation functions for artificial neurons.

Sigmoid Perceptrons

- The problem with LTU: step function is discontinuous, cannot use gradient descent
- Change the activation function (again): use a logistic (sigmoid) function

$$g(x) = \frac{e^x}{1 + e^x}$$



清华大学统计学研究中心

Sigmoid Perceptrons

- $g'(x) = g(x)\big(1 - g(x)\big)$

- Gradient descent (squared error loss):

$$w \leftarrow w - \alpha(g - y)g(1 - g)x$$

- Quiz: show the update formula for the **cross-entropy** loss:

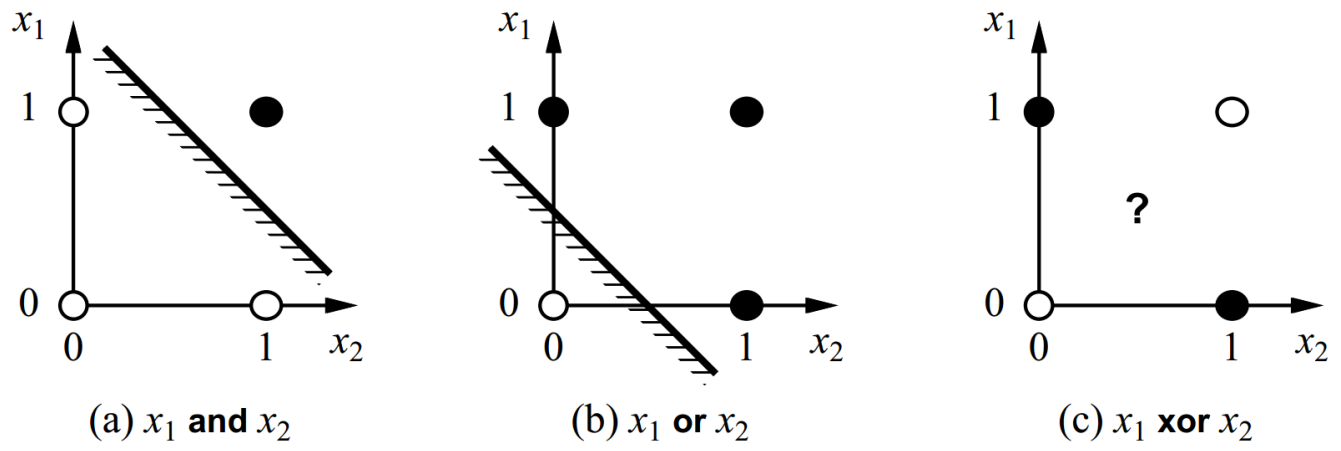
$$L = -y \log g - (1 - y) \log(1 - g)$$

- Cross-entropy for K-way classification: $L = -\sum_{k=1}^{K} y_k \log \hat{p}_k$.

清华大学统计学研究中心

Expressiveness of Perceptrons

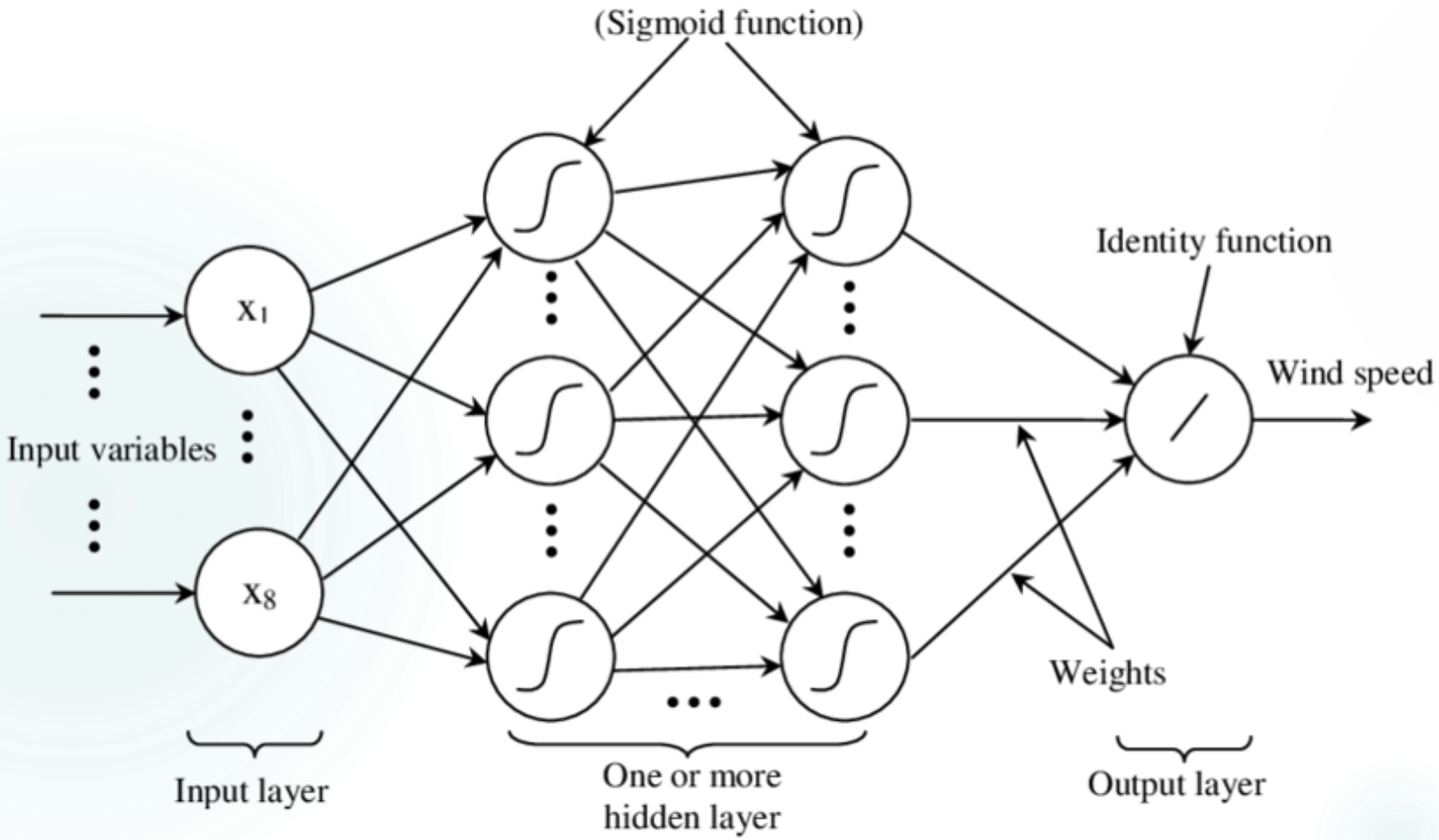
- The perceptron model was invented by Frank Rosenblatt in 1957. He optimistically predicted that the perceptron "may eventually be able to learn, make decisions, and translate languages".
- But, it was quickly proved that perceptrons could not be trained to recognize many classes of patterns, since even with a non-linear sigmoid function, the decision boundary a perceptron can produce is still linear.
- In 1969 a famous book entitled *Perceptrons* by Marvin Minsky and Seymour Papert showed that it was impossible for these classes of network to learn an XOR function.
- This contributed to the first AI winter.



清华大学统计学研究中心

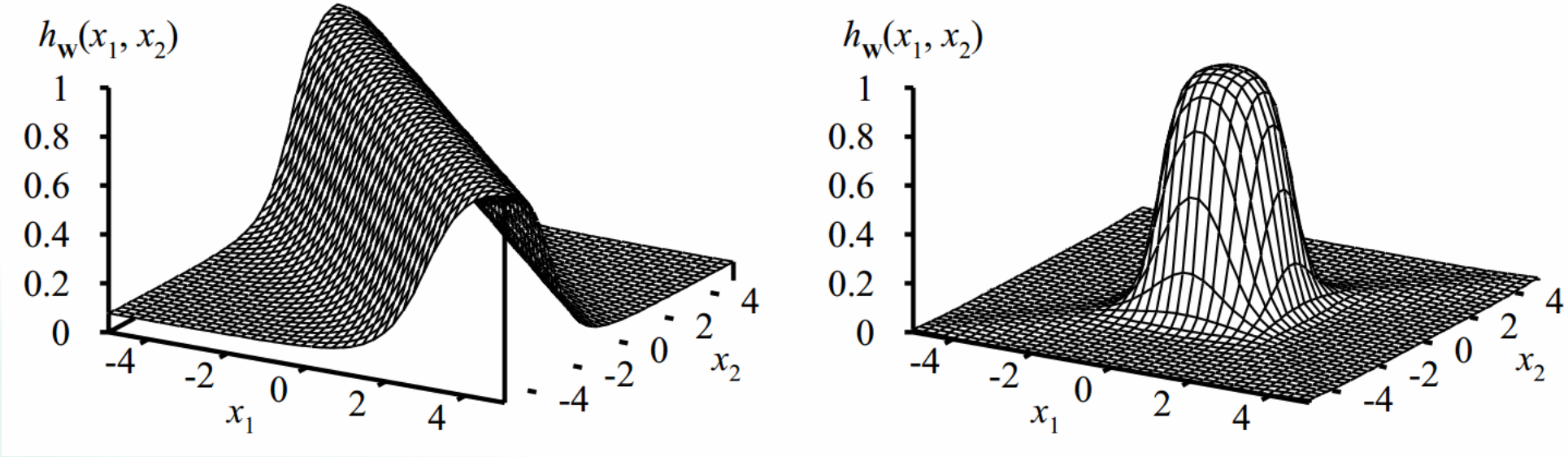
Multilayer Perceptron

- This is what we usually call a neural network.



清华大学统计学研究中心

Expressiveness of MLPs



- Combine two opposite-facing threshold functions to make a ridge
- Combine two perpendicular ridges to make a bump
- Add bumps of various sizes and locations to fit any surface

清华大学统计学研究中心

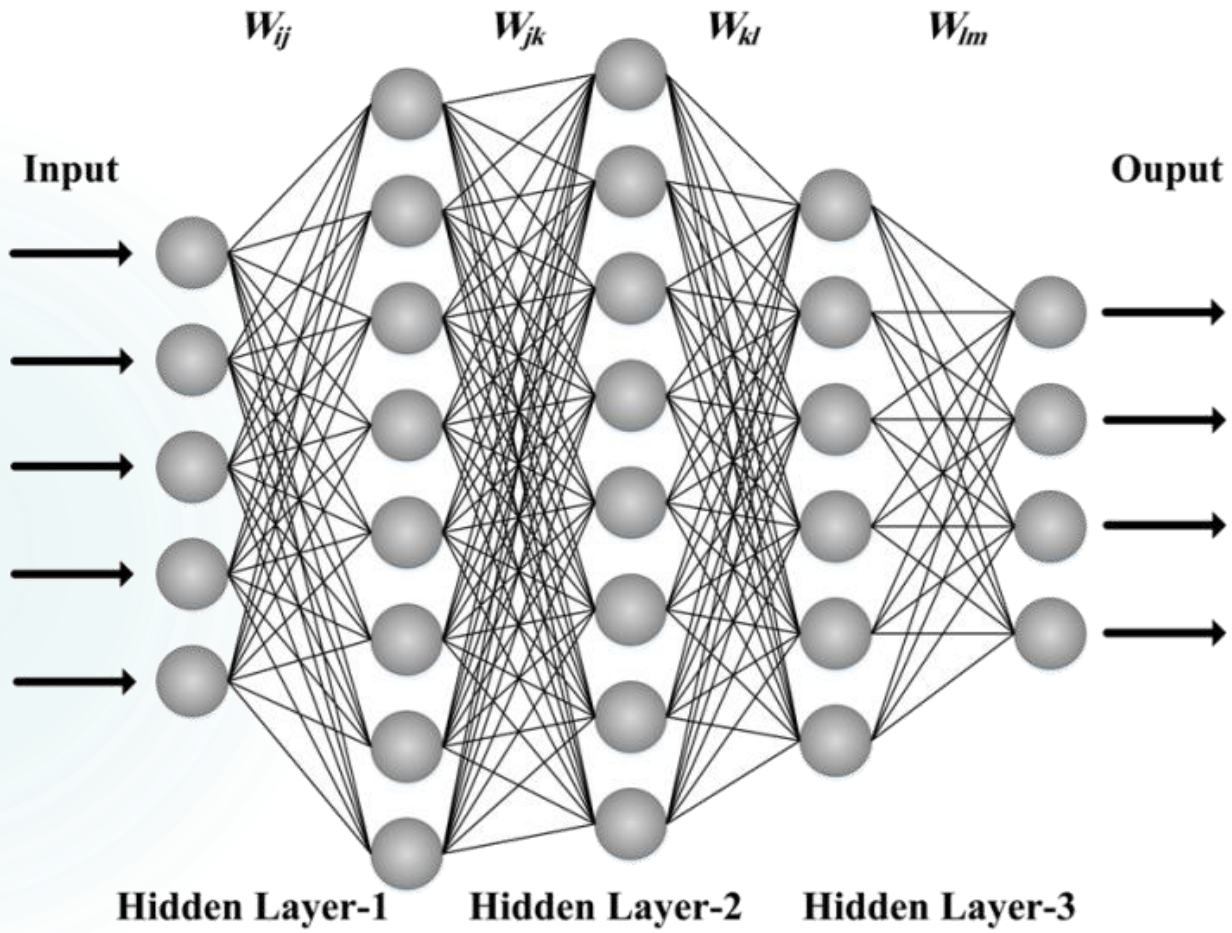
Expressiveness of MLPs

- *In theory*, we don't need too many layers:
- 1-hidden-layer net *with enough hidden units* can represent any continuous function of the inputs with arbitrary accuracy
- 2-hidden-layer net can even represent discontinuous functions

清华大学统计学研究中心

MLP for K-way Classification

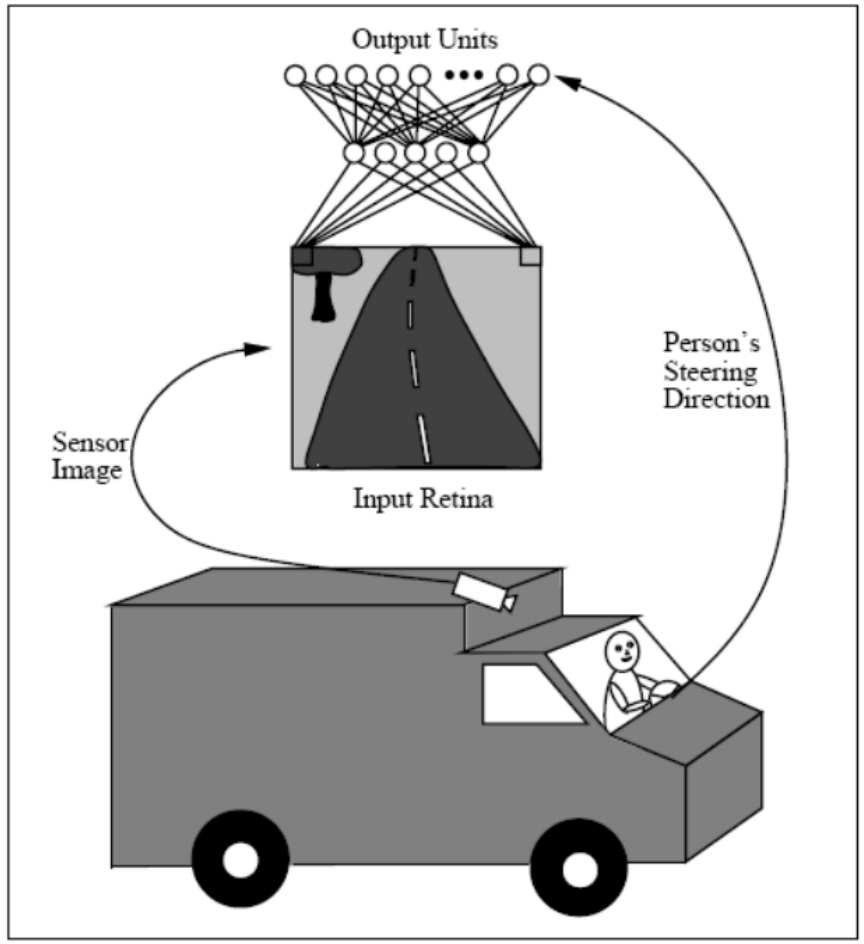
- Use K output units. Choose the class corresponding to the largest output.



清华大学统计学研究中心

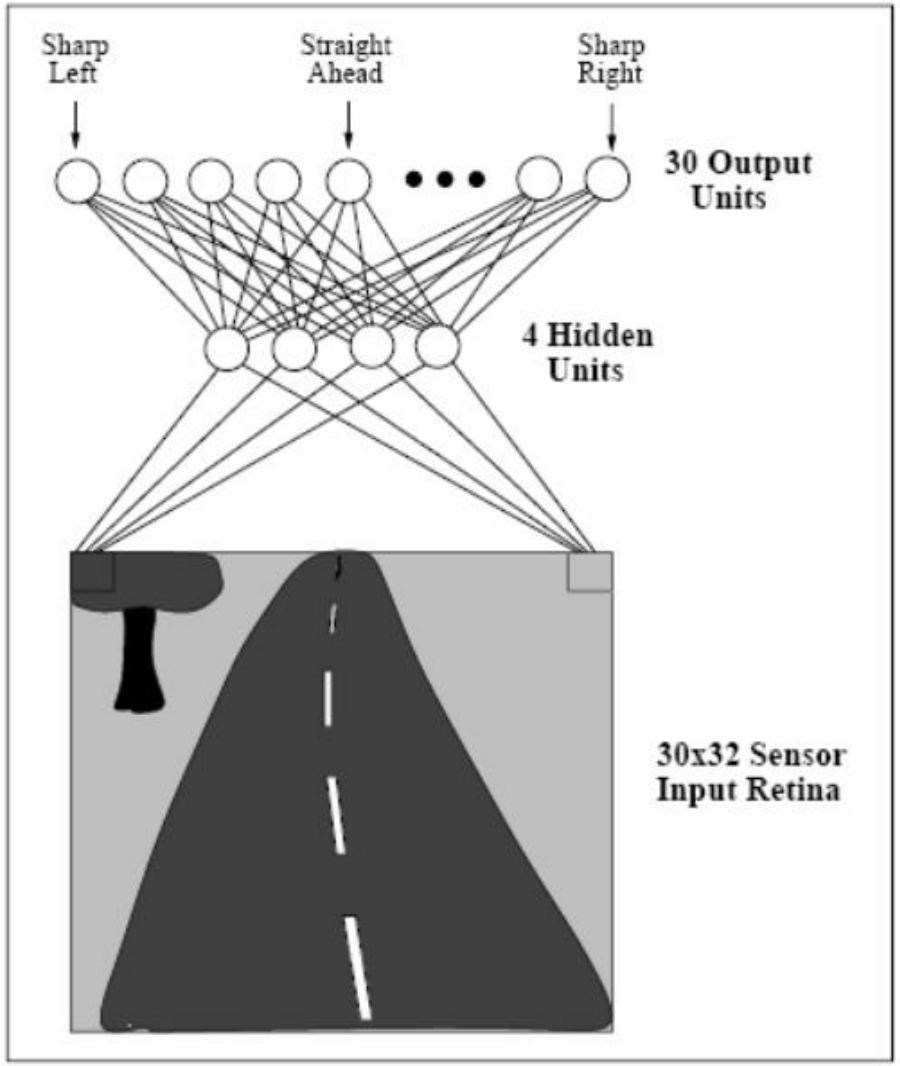
MLP for K-way Classification

Obtaining training data



[Pomerleau, 1995]

MLP for K-way Classification



[Pomerleau, 1995]

清华大学统计学研究中心

Today's Self-Driving



Training MLPs (Back-Propagation)

- Network setup
  - Let $x^{(0)}$ be the input vector.
  - Let $u^{(m)} = W^{(m-1)}x^{(m-1)} + b^{(m-1)}$, and let $x^{(m)} = g\left(u^{(m)}\right)$ be the values of Layer m, $m = 1, \ldots, M$, where g is the activation function, such as the sigmoid, tanh, and the ReLU:

$$g(x) = \begin{cases} \dfrac{e^x}{1+e^x} & \text{sigmoid activation,} \\ \dfrac{e^x - e^{-x}}{e^x + e^{-x}} & \text{tanh activation,} \\ \max(0, x) & \text{ReLU.} \end{cases}$$

清华大学统计学研究中心

Back-Propagation

$$g'(x) = \begin{cases} g(x)\big(1 - g(x)\big) & \text{sigmoid activation,} \\ 1 - g^2(x) & \text{tanh activation,} \\ I(x > 0) & \text{ReLU.} \end{cases}$$

- Layer $x^{(M)}$ is the output. Let L be the loss function based on $x^{(M)}$, such as the sum of squared error (SSE) or the cross entropy $-\sum_{k=1}^{K} y_k \log x_k^{(M)}$.

清华大学统计学研究中心

Back-Propagation

- Initialize the coefficients $W^{(m)}$ and $b^{(m)}, m = 0, \dots, M-1$, to some small random numbers.
- We use stochastic gradient descent to update the coefficients.
- For a training input $x^{(0)}$, compute $u^{(m)}$ and $x^{(m)}, m = 1, \dots, M$.
- Compute $\frac{\partial L}{\partial x^{(M)}}$.

清华大学统计学研究中心

Back-Propagation

- For the coefficients of Layer $M-1$, $W^{(M-1)}$ and $b^{(M-1)}$,

$$\frac{\partial L}{\partial W^{(M-1)}} = \left(\frac{\partial L}{\partial x^{(M)}} \times \nabla g\left(u^{(M)}\right)\right) \cdot \left(x^{(M-1)}\right)^T$$

and

$$\frac{\partial L}{\partial b^{(M-1)}} = \frac{\partial L}{\partial x^{(M)}} \times \nabla g\left(u^{(M)}\right),$$

where $\times$ means element-wise product.

清华大学统计学研究中心

Back-Propagation

- Also compute

$$\frac{\partial L}{\partial x^{(M-1)}} = \left(\left(\frac{\partial L}{\partial x^{(M)}} \times \nabla g\left(u^{(M)}\right) \right)^T W^{(M-1)} \right)^T$$

$$= \text{colSums}\left(W^{(M-1)} \times \left(\frac{\partial L}{\partial x^{(M)}} \times \nabla g\left(u^{(M)}\right) \right) \right).$$

清华大学统计学研究中心

Back-Propagation

- In the general case, when computing the gradients for Layer $m, m = 0, \dots, M-1$,

$$\frac{\partial L}{\partial W^{(m)}} = \left(\frac{\partial L}{\partial x^{(m+1)}} \times \nabla g\left(u^{(m+1)}\right) \right) \cdot \left(x^{(m)}\right)^T$$

and

$$\frac{\partial L}{\partial b^{(m)}} = \frac{\partial L}{\partial x^{(m+1)}} \times \nabla g\left(u^{(m+1)}\right).$$

清华大学统计学研究中心

Back-Propagation

- And

$$\frac{\partial L}{\partial x^{(m)}} = \left(\left(\frac{\partial L}{\partial x^{(m+1)}} \times \nabla g\left(u^{(m+1)}\right) \right)^T W^{(m)} \right)^T$$

$$= \text{colSums}\left(W^{(m)} \times \left(\frac{\partial L}{\partial x^{(m+1)}} \times \nabla g\left(u^{(m+1)}\right) \right) \right).$$

清华大学统计学研究中心

Back-Propagation

- When to terminate back-propagation? Overfitting and early stopping
  - After a fixed number of iterations (ok)
  - When training error less than a threshold (wrong)
  - When holdout set error starts to go up (ideal)
- The weights can converge to a local minimum
- Learning rate
  - Convergence sensitive to learning rate
  - Weight learning can be rather slow

清华大学统计学研究中心

Neural Network Parameter Tuning

- How many hidden units? How many layers? How to connect units?
  - Cross-validation
- Problem: It can be hard to have many layers (Why?)

清华大学统计学研究中心

Timeline

- 1946: The U.S. gov built the first electronic general-purpose computer ENIAC.
- 1947: William Shockley, John Bardeen and Walter Brattain invented the transistor.
- 1954: IBM introduced the IBM 704, the first mass-produced computer with floating-point arithmetic hardware.
- 1956: The FORTRAN programming language was born, developed for the 704.
- 1957: Rosenblatt's perceptrons were initially simulated on an IBM 704.
- 1958: Jack Kilby and Robert Noyce invented the integrated circuit, also known as the computer chip.

清华大学统计学研究中心

The Rises and Falls of Neural Networks

- Rosenblatt, 1957, invention of the Perceptron model.
- Minsky & Papert, 1969, pricked the neural network balloon.
- Middle '80s, MLPs and back-propagation.
- '90s, rise of SVM and other simpler methods.
  - A bet between Larry Jackel and Vladimir Vapnik (Bell Lab) in 1995:
  - Jackel: there would be a theoretical explanation to why large ANN would work by 2000.
  - Vapnik: by 2005 no body would use ANN any more.

清华大学统计学研究中心

1. Jackel bets (one fancy dinner) that by March 14, 2000 people will understand
quantitatively why big neural nets working on large databases are not so bad.
(Understanding means that there will be clear conditions and bounds)

Vapnik bets (one fancy dinner) that Jackel is wrong.

But .. If Vapnik figures out the bounds and conditions, Vapnik still wins the bet.

****

2. Vapnik bets (one fancy dinner) that by March 14, 2005, no one in his right mind will use neural nets that are essentially like those used in 1995.

Jackel bets (one fancy dinner) that Vapnik is wrong

3/14/95
V. Vapnik

3/14//95
L. Jackel

3/14/95
Witnessed by Y. LeCun





The Rises and Falls of Neural Networks

- The big comeback – **Deep Learning**.

